

Session initialization

Every BI session **must** start by importing `bf` and initializing the BI object. This single object is the gateway to everything: data handling, model definition, fitting, diagnostics, and saving.

Rule: Always import `bf` **before** importing `jax` or any other JAX-dependent module, so that the XLA configuration is applied first.

Python

```
from BayesForge import bf

m = bf(
    platform="cpu",          # "cpu", "gpu", or "tpu"
    cores=None,             # CPU cores (None = all available)
    float_precision=64,     # 64 for float64, 32 for float32
    rand_seed=42,           # int for reproducibility, True for random
    gpu_index=None,         # GPU index when platform="gpu"
    deallocate=False,       # deallocate existing device first
    print_devices_found=True, # print device info on init
)
```

Arguments

Argument	Type	Default	Description
<code>platform</code>	<code>str</code>	<code>"cpu"</code>	Hardware backend: <code>"cpu"</code> , <code>"gpu"</code> , or <code>"tpu"</code>
<code>cores</code>	<code>int</code>	<code>None</code>	CPU cores to allocate (<code>None</code> = all). Only for <code>platform="cpu"</code>

Argument	Type	Default	Description
<code>float_precision</code>	int or str	64	JAX floating-point precision. Pass 64 ("float64") or 32 ("float32")
<code>rand_seed</code>	int or bool	True	Reproducibility seed. Pass an int (e.g. 42) for reproducible results, or True for entropy-based randomness
<code>gpu_index</code>	int	None	Which GPU to use by index. Only for <code>platform="gpu"</code>
<code>deallocate</code>	bool	False	Deallocate existing device before setting up new configuration
<code>print_devices_found</code>	bool	True	Print detected devices on initialization
<code>backend</code>	str	"numpyro"	Inference backend: "numpyro" or "tfp"

Common initialization patterns

```
# Reproducible CPU session (default for development)
m = bf(platform="cpu", rand_seed=42, float_precision=64)

# GPU with specific device
m = bf(platform="gpu", gpu_index=0, rand_seed=42)

# Multi-core CPU for parallel chains
m = bf(platform="cpu", cores=8, rand_seed=42)

# 32-bit precision for memory efficiency
m = bf(platform="cpu", rand_seed=42, float_precision=32)
```

R

```
library(BayesianInference)
m = importBI(platform = "cpu", rand_seed = 42)
```

Julia

```
using BayesianInference
m = importBI(platform = "cpu", rand_seed = 42)
```

Saving and reusing a BI object

After fitting a model, save the complete BI object to disk. This persists the model, data, posteriors, and full sampler state — allowing you to reload and analyze later without re-fitting.

Save

```
m.save("/path/to/my_model.pkl")
# Default filename: "{model_name}_bf.pkl" in current directory
```

Load

```
m = bf.load("/path/to/my_model.pkl")
```

Once loaded, you can access:

<code>m.posteriors</code>	# Posterior samples
<code>m.posteriors_full</code>	# Full posterior (before any filtering)
<code>m.summary()</code>	# Posterior summary table
<code>m.data_on_model</code>	# Data used in the fit
<code>m.model</code>	# Model function
<code>m.diag</code>	# Diagnostic tools

Workflow with save/load

```

# --- Session 1: Fit and save ---
from BayesForge import bf
m = bf(platform="cpu", rand_seed=42)

def model(x, y):
    alpha = m.dist.normal(0, 10, name="alpha")
    beta  = m.dist.normal(0, 1, name="beta")
    sigma = m.dist.exponential(1, name="sigma")
    m.dist.normal(alpha + beta * x, sigma, obs=y)

m.fit(model, obs=dict(x=x, y=y))
m.save("/tmp/linear_regression.pkl")

# --- Session 2: Load and analyze (no re-fit) ---
from BayesForge import bf
m = bf.load("/tmp/linear_regression.pkl")
print(m.summary())
print(m.posteriors["beta"].mean())

```

Note: The object is serialized with `cloudpickle`, so the full model closure and sampler state are preserved. The saved `.pkl` file is self-contained and portable.